GNN-AoI: Correlation-Aware Scheduling of Real Sensor Networks

A graph neural network, trained with reinforcement learning, that decides which sensors may transmit under a hard Age-of-Information guarantee

Bhanu Prasad Palella Gullanki Bhagya Lakshmi Pavna Preethikha Madhumitha S

Amrita School of Artificial Intelligence
Guide: [guide's name and designation]

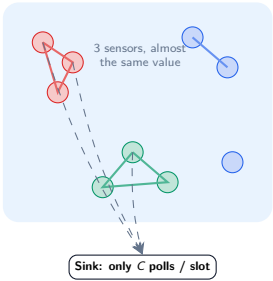
Final Project Presentation, [presentation date]

Table of Contents

1. Introduction
2. Literature Review
3. Methodology
4. Implementation
5. Results and Discussion
6. Conclusion and Future Work
7. References

Problem Statement

- A sink must track the readings of M **sensors** (temperature, traffic speed, ...) over a shared wireless channel.
- Only $C \ll M$ **sensors may transmit per slot**: the sink must choose *which*.
- Classic schedulers minimise the **Age of Information** (AoI) and poll whoever is stalest, ignoring *content*: correlated neighbours are polled as if independent.
- This wastes capacity on **redundant data** and worsens the sink's estimate of the field.



Polling three correlated neighbours tells the sink little more than polling one of them.

Question

With only C polls per slot, which sensors should transmit so that the sink's estimate of *all* M readings is as accurate as possible, **without letting any sensor go stale**?

Objectives

1. **Model** the sink's knowledge with a Kalman filter on a correlation graph *learned from data*, so uncertainty about a sensor drops when its neighbours report.
2. **Design** a GNN scheduler (message passing over the correlation graph and the sink's posterior) that picks C sensors per slot, trained end-to-end with **PPO** on the true estimation error.
3. **Guarantee freshness**: no sensor's AoI may exceed a bound F, enforced by construction for every policy, not as a soft penalty.
4. **Validate on real data**: 45-sensor Intel Berkeley Lab temperatures and 325-sensor PEMS-BAY freeway speeds, evaluated only on held-out test periods.
5. **Compare** against AoI-based, correlation-heuristic and model-based (Kalman-greedy) schedulers with significance tests, plus an ablation of the message passing.
6. **Demonstrate** the schedulers live on real recordings in a browser front end.

Literature Review

S.No	Journal / Conference, Publisher (Year)	Technique adopted	Inference	Cons
1	Tripathi & Modiano [1]: ACM MobiHoc (2022); ext. IEEE/ACM ToN (2024)	Randomised and Lyapunov max-weight policies for weighted AoI of correlated sources	Both within a factor 2 of optimal; correlation improves AoI scaling	Abstract correlation model; AoI, not estimation error
2	Fidler, Gallistl, Champati & Widmer [5]: IEEE Trans. Commun. (2026)	2D-AoI: spatial distance mapped to an equivalent age (Gaussian kernels)	Distance acts as an age offset; values correlated samples	A metric, not a scheduler; known kernel assumed
3	Chen, Liu, Quevedo, Li & Vucetic [8]: IEEE ICC (2023)	Structure-enhanced DRL for multi-sensor remote-estimation MSE	10–25% lower MSE and 50% less training time than benchmark DRL	Needs analytically derived policy structure; no learned correlation graph
4	Reyhan & Coleri [9]: IEEE Trans. Commun. (2025)	Optimisation theory + safe DRL (teacher–student) with Peak-AoI constraints	Constraint-satisfying DRL converges faster and more stably	Peak AoI bounded only in probability; optimises power, not error
5	Zhao, Verma, Rao, Swami & Segarra [11]: IEEE Trans. Wireless Commun. (2023)	GCN-guided tree search / distributed greedy for link scheduling; RL training	Shallow GCN halves the sub-optimality gap of distributed greedy	Throughput utility; graph = conflicts, not data correlation
6	Wang, Wang, Chen & Zhou [12]: IEEE/IFIP WiOpt (2024)	Age-aware randomised policy; per-slot Lyapunov drift solved by an unsupervised MPNN	Permutation-equivariant MPNN scales and lowers AoI vs. baselines	AoI only (content-agnostic); graph encodes interference

Further related work: [3,4,6,7,10,13]. Foundations: AoI [15], max-age scheduling [16], AoI survey [14], DRL sensor scheduling [18].

Base Paper

R. V. Ramakanth, V. Tripathi and E. Modiano [2]

"Monitoring Correlated Sources: AoI-based Scheduling is Nearly Optimal," *IEEE INFOCOM 2024*, pp. 641–650; extended in *IEEE Trans. Mobile Computing*, vol. 24, no. 2, 2025.

WHAT THE PAPER DOES

- Correlated sources modelled as a discrete-time Wiener process with multivariate Gaussian increments (general covariance).
- The monitor estimates all sources with a **Kalman filter**; only **one source** can be observed per slot.
- Shows the error of any policy that ignores instantaneous error is bounded above and below by a **weighted sum of AoI**.
- Result: AoI-based scheduling that *ignores correlation* is within a constant factor of optimal; optimal error grows roughly with the square of the number of sources.

HOW OUR PROJECT BUILDS ON IT

- **Same estimation framework**: a Kalman-filter sink tracking correlated sources.
- **Their policy class is our baseline**: Max-AoI (B1) is the unweighted AoI-based scheduler.
- **Extensions**: $C > 1$ polls per slot, stationary AR(1) model and correlation *learned from data*, hard freshness bound, learned GNN policy, real sensor datasets.
- **Finding**: the constant-factor gap matters in practice: correlation-aware learned scheduling lowers the error by **20%** on Intel Lab and **2.4%** on PEMS-BAY versus AoI-based scheduling.

Research Gap and Novelty

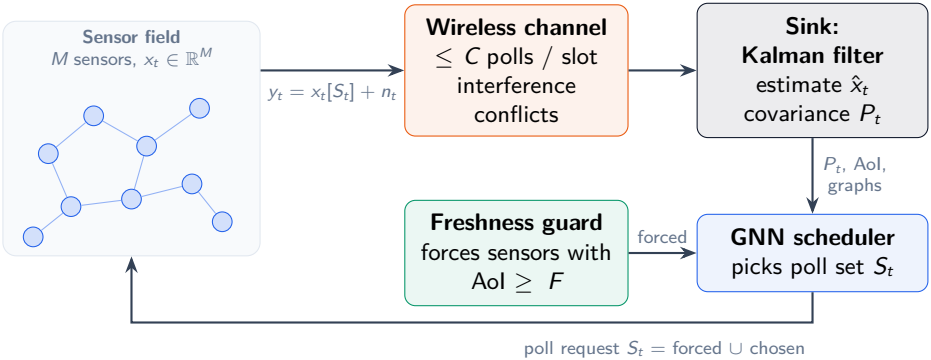
RESEARCH GAP

- **Correlation-aware AoI work** [1–5] is analytical: hand-specified correlation models, often one source per slot, and no learned policy.
- **Learned schedulers** (DRL [6–9], GNN [10–13]) optimise AoI, throughput or power; their graphs describe *interference*, not *what the data says*.
- Freshness is usually a **soft or probabilistic** constraint [7,9].
- Few works test on **real multi-sensor recordings** with a strict train/test split in time.

NOVELTY OF THIS WORK

- ✔ **Correlation graph learned from data**, used twice: as a GNN graph and inside the sink's Kalman filter.
- ✔ **Two graphs per slot**: static correlation A_{corr} and the sink's live posterior correlation A_{post}.
- ✔ **Residual pointer decoder** initialised to Kalman-greedy; PPO only learns where looking ahead beats greedy.
- ✔ **Hard AoI guarantee** for every policy (forced polls).
- ✔ **Real data**: Intel Lab (45) and PEMS-BAY (325), held-out test periods, Wilcoxon tests, plus a live demo.

Methodology (1/5): System Model and Overall Pipeline



Pipeline: training data → fit AR(1) + innovation covariance → correlation graph → simulate / replay → PPO-train GNN → evaluate on held-out test period → live demo

Methodology (2/5): Correlation Graph and Kalman Sink

FIELD MODEL, FITTED ON THE TRAINING PERIOD ONLY

$$x_t = \mathrm{diag}(\rho)\, x_{t-1} + w_t, \qquad w_t \sim \mathcal{N}(0, Q)$$
$$y_t = x_t[S_t] + n_t, \qquad n_t \sim \mathcal{N}(0, rI)$$

- ρ_i: least squares on one-step changes, per sensor
- Q: covariance of the residuals (innovations)

CORRELATION GRAPH

$$(A_{\mathrm{corr}})_{ij} = |\hat{r}_{ij}| \cdot \mathbf{1}\big[\,|\hat{r}_{ij}| \geq 0.3\,\big]$$

$\hat{r}$ = innovation correlation. Edges link sensors whose *surprises* move together.

SINK: KALMAN FILTER

$$P_t^- = D\,P_{t-1}\,D + Q, \qquad D = \mathrm{diag}(\rho)$$
$$P_t = P_t^- - P^-_{:,S}\big(P^-_{SS} + rI\big)^{-1} P^-_{S,:}$$

Why this matters

Polling sensor j lowers P_{ii} for every *correlated* neighbour i. The variance removed by polling i next is

$$g_i = \frac{\sum_k P_{ki}^2}{P_{ii} + r},$$

which drives both the Kalman-greedy baseline [19] and our decoder.

Methodology (3/5): Formulation and Freshness Guarantee

OBJECTIVE

$$\min_{\pi} \ \mathbb{E}\left[\frac{1}{TM}\sum_{t=1}^{T}\|x_t - \hat{x}_t\|^2\right]$$

SUBJECT TO, IN EVERY SLOT T

- capacity: $|S_t| \le C$
- interference: no two conflicting sensors in S_t
- **freshness**: $a_i(t) \le F$ for every sensor i

AGE OF INFORMATION

$$a_i(t+1) = \begin{cases} 1 & i \in S_t \\ a_i(t) + 1 & \text{otherwise} \end{cases}$$

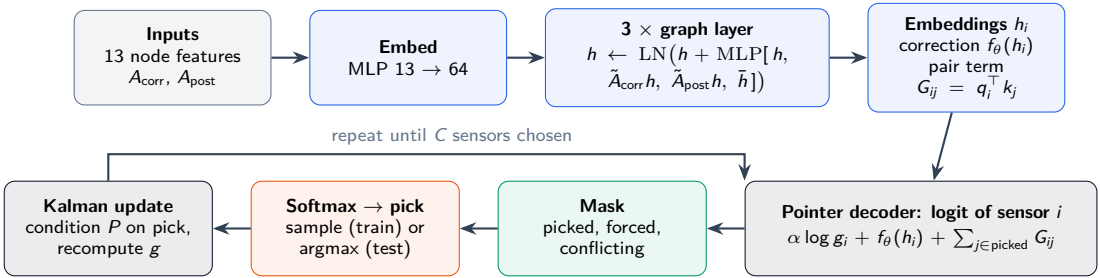
FRESHNESS GUARD (HARD, EVERY POLICY)

1. Forced set $\mathcal{F}_t = \{i : a_i(t) \ge F\}$, oldest first, conflict-feasible.
2. Forced sensors are committed *before* the policy acts.
3. The policy fills the remaining $C - |\mathcal{F}_t|$ slots.
4. The environment asserts capacity, conflicts and freshness each step.

Setting

$F = \lfloor 3M/C \rfloor$ (Intel: 22 slots = 1.8 h; PEMS-BAY: 24 slots = 2 h). The policy is a Markov decision process over the sink's state, solved with PPO.

Methodology (4/5): GNN Scheduler Architecture



- Features: AoI, last polled, posterior variance (and log), gain g_i (and log), forced flag, freshness slack, C/M, $\hat{\rho}_i$, drift rate, graph degree, guard on/off.
- $A_{\text{post}} = |P_{ij}| / \sqrt{P_{ii} P_{jj}}$: who is redundant *right now*.
- **Residual design**: f_θ and G start at zero, so the untrained actor *is* Kalman-greedy (unit-tested).
- Plackett–Luce product of softmaxes gives exact log-probabilities for PPO.
- 137k parameters; works for any M (shared weights).

Methodology (5/5): Training with PPO

Algorithm 1 PPO training of the GNN scheduler

Initialise actor π_θ = Kalman-greedy (f_θ=G=0), critic V_ϕ
for iteration $= 1, \ldots, N$ **do**
  Sample batch of envs: synthetic ($M \in [10, 100]$, random C, layout, ρ, F) or, with prob. 0.35, real training-period replays
  for $t = 1, \ldots, 96$ **do**
    $\mathcal{F}_t \leftarrow$ freshness guard; $S_t \leftarrow \mathcal{F}_t \cup \pi_\theta(\text{features}, A_{\text{corr}}, A_{\text{post}})$
    Step env; reward $r_t = -(1-\gamma)\log(\text{error}_t + 0.005)$
  end for
  Advantages $\hat{A}_t$ with GAE ($\gamma = 0.95$, $\lambda = 0.9$)
  for 4 epochs, minibatches of 512 **do**
    $\eta_t = \pi_\theta(S_t)/\pi_{\theta_{\text{old}}}(S_t)$ ▷ Plackett–Luce
    $L = -\min(\eta_t \hat{A}_t, \text{clip}(\eta_t, 0.85, 1.15)\hat{A}_t) - 0.002\,\mathcal{H}$
    Adam step on θ; regress V_ϕ to returns
  end for
  Every 10 iters: validate; keep best checkpoint
end for

WHY THESE CHOICES

- **Greedy start**: PPO begins from the strongest model-based scheduler instead of random.
- **Log reward** balances small and large networks in one batch.
- **Real replays**: the sink is fitted on the first 70% of the training period and episodes run on the next 20%, so the policy learns to cope with model mismatch.
- Last 10% of training data + seeds ≥ 9000 for validation; test periods and layouts (≥ 10000) are never seen.

Implementation (1/5): Tech Stack and Code Structure

STACK

- **Python 3.13**, NumPy, PyTorch (CPU)
- Dense batched message passing (no graph library needed at $M \leq 325$)
- Matplotlib for figures and the animated GIF
- Standard-library HTTP server + vanilla JavaScript/Canvas front end
- pytest (21 tests) on GitHub Actions CI
- Open source on GitHub:
  `BhanuPrasadPalella-01/gnn-aoi-scheduler`

REPOSITORY LAYOUT

```
gnn_aoi/
  simulator.py     correlated field, fit_model
  datasets.py      Intel Lab, PEMS-BAY loaders
  graphs.py        correlation & interference
  estimator.py     Kalman sink
  env.py           slots, AoI, freshness guard
  policies/        5 baselines + GNN policy
  gnn/             features, model, PPO train
  experiments.py   benchmarks, sweeps, stats
  live.py          step-by-step live session
  server.py        HTTP API for the demo
frontend/          live dashboard (JS/Canvas)
tests/             21 unit/regression tests
```

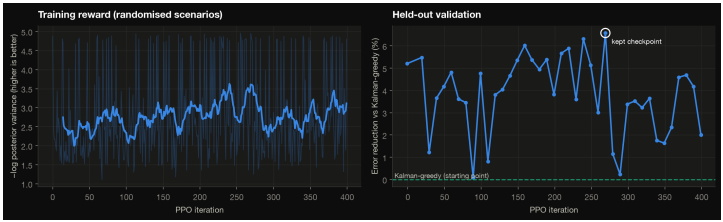
Implementation (2/5): Real Datasets and Preprocessing

	Intel Berkeley Research Lab [25]	**PEMS-BAY** [24]
Quantity	Indoor temperature (°C)	Freeway traffic speed (mph)
Sensors M	45 (after quality filtering)	325 loop detectors
Slot	5 min	5 min
Capacity C / bound F	6 polls / 22 slots	40 polls / 24 slots
Train period	28 Feb – 8 Mar 2004	Jan – Apr 2017
Test period	9 – 18 Mar 2004 (10 windows × 200 slots)	May – Jun 2017 (15 windows × 288 slots)
Daily profile removed	time of day	time of day, weekday / weekend

- Standardised with training-period statistics only; no test data leaks into the model (unit-tested).
- Intel data has **recording outages** (10–11 and 15–16 Mar): episodes never cross a gap.
- float32 storage and pre-fitted models cut memory from ≈1.75 GB to ≈200–400 MB.

Implementation (3/5): Training Setup

Hyper-parameter	Value
Embedding size / layers	64 / 3
Envs per batch	32 (at M=40)
Rollout length	96 slots
PPO iterations	400
γ / GAE λ	0.95 / 0.9
Clip ϵ	0.15
Epochs / minibatch	4 / 512
LR actor / critic	10^{-4} / 5×10^{-4}
Entropy coef.	0.002
Real-data share	35%
Hardware	laptop CPU
Training time	≈35 min



- Left: training reward over randomised scenarios (noisy by design: every batch is a different network).
- Right: held-out validation error vs. Kalman-greedy, which is exactly where training starts (0%). The best checkpoint is kept (**+6.6%** better than Kalman-greedy).
- Training is resumable and was warm-started from an earlier run.

Implementation (4/5): Baselines and Evaluation Protocol

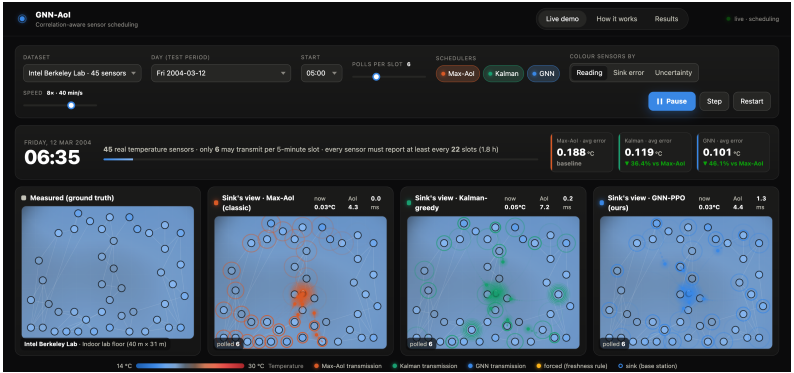
SCHEDULERS (SAME FRESHNESS GUARD FOR ALL)

Max-AoI (B1)	poll the stalest sensors: AoI-based scheduling [2,16]
Random	uniform choice
Corr-diverse (v4)	stalest first, discount neighbours by |correlation|
Max-uncertainty	largest posterior variance P_{ii}
Kalman-greedy	greedy variance reduction on the sink's covariance [19]
Oracle-greedy	Kalman-greedy with the *true* covariance (synthetic only)
Ablation	GNN with message passing removed

PROTOCOL

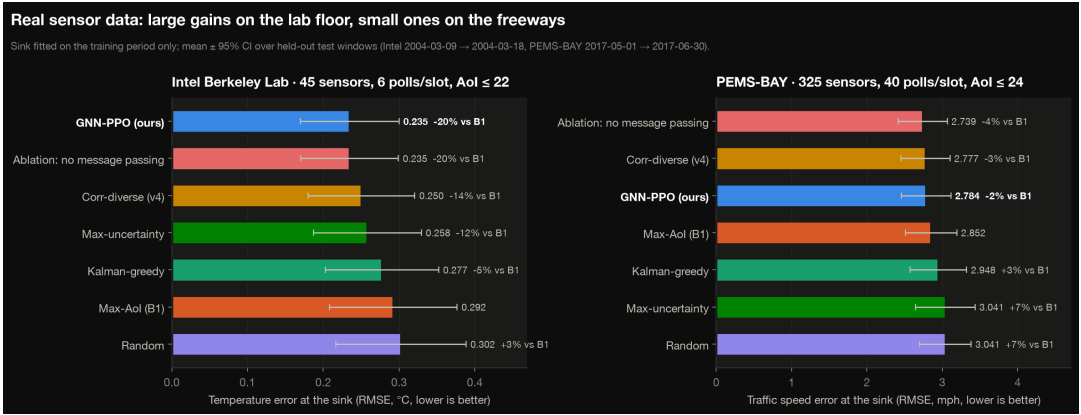
- **Metric**: sink estimation error (RMSE in °C / mph; MSE in standardised units).
- **Real data**: held-out test windows only.
- **Synthetic**: 30 unseen layouts $\times$ 350 slots, uniform and clustered.
- **Statistics**: mean $\pm$ 95% CI, per-window win rate, Wilcoxon signed-rank test.
- Sweeps over capacity C, network size M, interference, freshness on/off.
- Decision time per slot measured on CPU.

Implementation (5/5): Live Transmission Demo



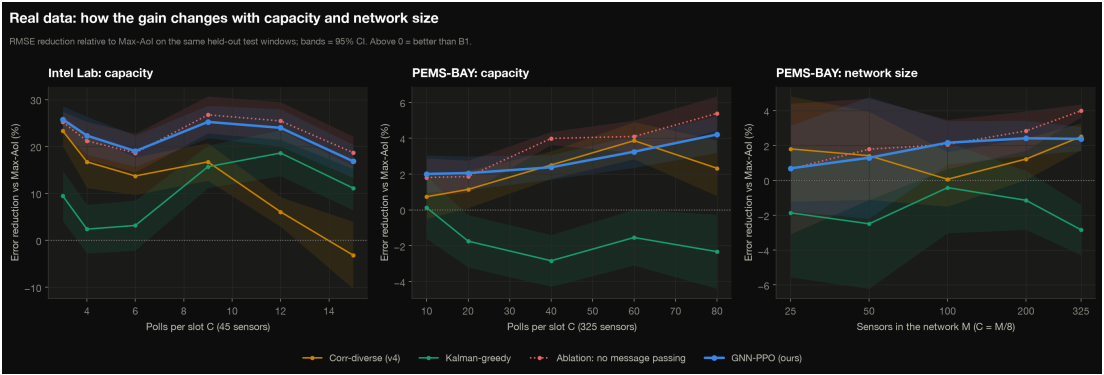
- Replays a **real test day** slot by slot (here Intel Lab, 12 Mar 2004).
- Three schedulers run side by side on the same readings; rings = sensors polled this slot.
- Each map shows the **sink's view**: what it believes the field looks like.
- "Why these sensors?" shows the GNN's pick probabilities; the log lists every transmission.
- Runs locally: `python -m gnn_aoi serve`, then open `localhost:8000`.

Results (1/5): Real Sensor Data, Held-out Test Periods



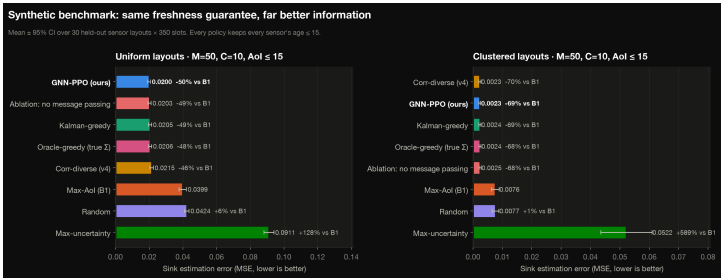
- **Intel Lab**: GNN **0.235 °C** vs. Max-AoI 0.292 (-20%) and Kalman-greedy 0.277. GNN beats Kalman-greedy in **10/10** test windows ($p = 0.002$).
- **PEMS-BAY**: GNN 2.78 mph vs. Max-AoI 2.85 (-2.4%); Kalman-greedy is **worse** than Max-AoI here (2.95). GNN beats Kalman-greedy in **15/15** windows ($p = 6 \times 10^{-5}$).

Results (2/5): Capacity and Size Sweeps, Real Data



- Intel Lab: GNN stays **17–26%** below Max-AoI for every capacity $C = 3 \ldots 15$; Kalman-greedy and the correlation heuristic are inconsistent (the heuristic even loses at $C = 15$).
- PEMS-BAY: the gain grows with capacity (2% $\rightarrow$ 4%) and network size; Kalman-greedy is **below** Max-AoI for most settings because its fitted model is over-confident on freeway data.

Results (3/5): Synthetic Data, Ablation, Freshness



- Uniform: GNN -50% MSE vs. Max-AoI; beats Kalman-greedy on 73% of layouts ($p = 0.008$) and **oracle-greedy with the true covariance** on 70% ($p = 0.003$).
- Clustered: all correlation-aware policies tie ($\approx -69\%$ vs. Max-AoI).
- Ablation: message passing helps on synthetic layouts (wins 67% uniform, 87% clustered).

FRESHNESS GUARD SWITCHED OFF (M=50, C=10)

Policy	MSE	Starved	Max AoI
Max-AoI	0.0399	0	5
Kalman-greedy	0.0229	**18**	**401**
Oracle-greedy	0.0226	**19.6**	**401**
GNN (ours)	**0.0200**	**0**	10.3

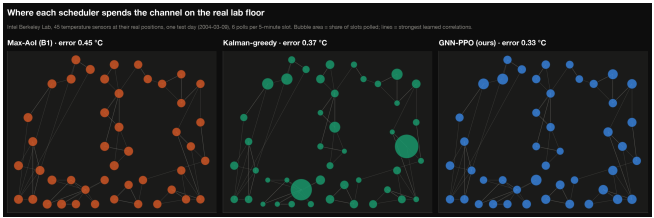
Greedy variance reduction **abandons** sensors it can infer from neighbours (never polled in 400 slots). The GNN gets lower error *and* polls everyone, even without the guard.

Results (4/5): Comparison with Recent Literature

Work	Approach	Reported in their own setting	Same approach, measured on our real data
Ramakanth *et al.* [2], INFOCOM 2024; Kadota *et al.* [16]	AoI-based (max-age) scheduling, correlation ignored	Provably within a constant factor of optimal (Wiener model, 1 source / slot)	Our Max-AoI (B1): GNN lowers RMSE **20%** (Intel) and **2.4%** (PEMS-BAY); wins 10/10 and 15/15 windows
Chen *et al.* [8], ICC 2023	Structure-enhanced DRL for remote-estimation MSE	10–25% lower MSE than benchmark DRL; 50% less training time	GNN: **25.8%** lower MSE than Kalman-greedy (Intel), **13.4%** (PEMS-BAY); trains in $\approx$35 min on a CPU
Krause *et al.* [19] (greedy variance reduction)	Submodular greedy sensor selection	Near-optimal one-step selection	GNN beats it in 73% of synthetic layouts and beats it *with the true covariance* in 70%
Reyhan & Coleri [9], TCOM 2025; Farag *et al.* [7]	DRL with AoI-violation *probability* constraints	Constraints satisfied in probability	**Hard** bound: zero violations by construction; unconstrained greedy starves 18/50 sensors

Numbers in the third column come from the cited papers' own setups and are not directly comparable; the fourth column re-runs that *class* of method on our held-out data under identical conditions.

Results (5/5): Discussion and Limitations



Bubble area = share of slots each sensor was polled (Intel Lab, one test day). Max-AoI is uniform; Kalman-greedy concentrates on a few "hub" sensors; the GNN spreads polls by redundancy.

WHAT THE RESULTS SAY

- Correlation awareness pays most where sensors are **strongly redundant** (lab floor), less on freeways.
- The learned policy is **robust to model mismatch**: Kalman-greedy trusts a wrong model; PPO on real replays corrects for it.
- Fast: 4.6 ms (Intel) and 37 ms (PEMS-BAY) per decision vs. a 5-minute slot.

LIMITATIONS (HONEST)

- On real data the **no-message-passing ablation ties** the full GNN: the gain comes mostly from the learned residual decoder.
- Synthetic $M \geq 100$: Kalman-greedy is 8–12% better (training covered $M \leq 100$).
- Linear-Gaussian sink; ideal channel apart from capacity and conflicts.

Conclusion

- Built **GNN-AoI**: a scheduler that learns the sensors' correlation graph from data, tracks the field with a Kalman sink, and picks transmissions with a GNN pointer decoder trained by PPO.
- Every sensor stays fresh by construction (hard AoI bound), unlike probabilistic or penalty-based methods.
- On **real held-out data** it lowers the sink's error by **20%** on 45 Intel Lab sensors and **2.4%** on 325 PEMS-BAY sensors versus AoI-based scheduling, and beats Kalman-greedy in **every** test window on both datasets.
- On synthetic fields it halves the error of max-age scheduling and even beats greedy selection that knows the true covariance.
- Delivered as an open-source, tested package with reproducible experiments and a live browser demo of the transmissions.

Future Work

- **Lossy channels**: packet errors and retransmissions (HARQ, as in [6]).
- **Online adaptation**: re-fit the correlation graph as the field drifts (seasons, new sensors).
- **Bigger networks**: train on $M > 100$ and use sparse message passing for thousands of sensors.
- **Non-linear sinks**: replace the Kalman filter with a learned spatio-temporal predictor (e.g. graph-recurrent models [24]).
- **Distributed scheduling**: each gateway decides locally with the same shared GNN.
- **Hardware pilot**: run the scheduler on a real LoRa / Zigbee testbed.

References (1/3)

[1] V. Tripathi and E. Modiano, "Optimizing age of information with correlated sources," in *Proc. ACM MobiHoc*, 2022, pp. 41–50; extended in *IEEE/ACM Trans. Netw.*, vol. 32, no. 6, pp. 4660–4675, 2024.

[2] R. V. Ramakanth, V. Tripathi, and E. Modiano, "Monitoring correlated sources: AoI-based scheduling is nearly optimal," in *Proc. IEEE INFOCOM*, 2024, pp. 641–650; extended in *IEEE Trans. Mobile Comput.*, vol. 24, no. 2, pp. 1043–1054, 2025.

[3] E. Erbayat, A. Maatouk, P. Zou, and S. Subramaniam, "Age of information optimization and state error analysis for correlated multi-process multi-sensor systems," in *Proc. ACM MobiHoc*, 2024, pp. 331–340.

[4] E. Erbayat, A. Maatouk, P. Zou, and S. Subramaniam, "Age of information optimization with preemption strategies for correlated systems," in *Proc. IEEE ISIT*, 2025.

[5] M. Fidler, F. Gallistl, J. P. Champati, and J. Widmer, "2D-AoI: Age-of-information of distributed sensors for spatio-temporal processes," *IEEE Trans. Commun.*, vol. 74, pp. 645–661, 2026.

[6] E. T. Ceran, D. Gündüz, and A. György, "A reinforcement learning approach to age of information in multi-user networks with HARQ," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 5, pp. 1412–1426, 2021.

[7] H. Farag, M. Gidlund, and Č. Stefanović, "A deep reinforcement learning approach for improving age of information in mission-critical IoT," in *Proc. IEEE GCAIoT*, 2021, pp. 14–18.

[8] J. Chen, W. Liu, D. E. Quevedo, Y. Li, and B. Vucetic, "Structure-enhanced deep reinforcement learning for optimal transmission scheduling," in *Proc. IEEE ICC*, 2023, pp. 1212–1218.

References (2/3)

[9] B. G. Reyhan and S. Coleri, "Safe deep reinforcement learning for resource allocation with peak age of information violation guarantees," *IEEE Trans. Commun.*, vol. 73, no. 12, pp. 14197–14211, 2025.

[10] Y. Shen, Y. Shi, J. Zhang, and K. B. Letaief, "Graph neural networks for scalable radio resource management: Architecture design and theoretical analysis," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 1, pp. 101–115, 2021.

[11] Z. Zhao, G. Verma, C. Rao, A. Swami, and S. Segarra, "Link scheduling using graph neural networks," *IEEE Trans. Wireless Commun.*, vol. 22, no. 6, pp. 3997–4012, 2023.

[12] L. Wang, Q. Wang, H. Chen, and S. Zhou, "Age of information-oriented probabilistic link scheduling for device-to-device networks," in *Proc. IEEE/IFIP WiOpt*, 2024 (arXiv:2410.20196).

[13] W. Wang, Q. Wu, P. Fan, N. Cheng, W. Chen, J. Wang, and K. B. Letaief, "Optimizing age of information in vehicular edge computing with federated graph neural network multi-agent reinforcement learning," *IEEE Trans. Wireless Commun.*, vol. 25, pp. 23017–23032, 2026.

[14] R. D. Yates, Y. Sun, D. R. Brown, S. K. Kaul, E. Modiano, and S. Ulukus, "Age of information: An introduction and survey," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 5, pp. 1183–1210, 2021.

[15] S. Kaul, R. Yates, and M. Gruteser, "Real-time status: How often should one update?" in *Proc. IEEE INFOCOM*, 2012, pp. 2731–2735.

[16] I. Kadota, A. Sinha, E. Uysal-Biyikoglu, R. Singh, and E. Modiano, "Scheduling policies for minimizing age of information in broadcast wireless networks," *IEEE/ACM Trans. Netw.*, vol. 26, no. 6, pp. 2637–2650, 2018.

[17] A. Maatouk, S. Kriouile, M. Assaad, and A. Ephremides, "The age of incorrect information: A new performance metric for status updates," *IEEE/ACM Trans. Netw.*, vol. 28, no. 5, pp. 2215–2228, 2020.

References (3/3)

[18] A. S. Leong, A. Ramaswamy, D. E. Quevedo, H. Karl, and L. Shi, "Deep reinforcement learning for wireless sensor scheduling in cyber–physical systems," *Automatica*, vol. 113, art. 108759, 2020.

[19] A. Krause, A. Singh, and C. Guestrin, "Near-optimal sensor placements in Gaussian processes: Theory, efficient algorithms and empirical studies," *J. Mach. Learn. Res.*, vol. 9, pp. 235–284, 2008.

[20] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Trans. ASME J. Basic Eng.*, vol. 82, no. 1, pp. 35–45, 1960.

[21] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. ICLR*, 2017.

[22] W. Kool, H. van Hoof, and M. Welling, "Attention, learn to solve routing problems!" in *Proc. ICLR*, 2019.

[23] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[24] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," in *Proc. ICLR*, 2018 (PEMS-BAY dataset).

[25] P. Bodik, W. Hong, C. Guestrin, S. Madden, M. Paskin, and R. Thibaux, "Intel Lab Data," 2004. [Online]. Available: `db.csail.mit.edu/labdata/labdata.html`

Thank You

Questions & Answers

Bhanu Prasad Palella · Gullanki Bhagya Lakshmi · Pavna Preethikha · Madhumitha S

Amrita School of Artificial Intelligence

`github.com/BhanuPrasadPalella-01/gnn-aoi-scheduler`

Live demo: `python -m gnn_aoi serve` → `localhost:8000`